

Reti Neurali con Keras

Dataset Iris — EDA e Classificazione

Andrea Morlotti · Gianluca Cerea

2025 – 2026



Indice

1. Il Dataset Iris
2. Analisi Esplorativa (EDA)
3. Preprocessing
4. Teoria delle Reti Neurali
5. Funzioni di Attivazione
6. Optimizer
7. Loss Function e Metriche
8. Costruzione del Modello con Keras
9. Training e Risultati
10. Confronto SGD vs Adam

II Dataset Iris

Ronald Fisher, 1936

- 150 campioni totali
- 3 classi: Setosa, Versicolor, Virginica (50 ciascuna)
- 4 feature (cm): sepal length/width, petal length/width
- Obiettivo: classificare la specie dalla misurazione

Perche Iris per le reti neurali?

Bilanciato (50 per classe) — nessun bias

Setosa linearmente separabile dalle altre due

Versicolor/Virginica si sovrappongono — serve non-linearita

Piccolo — training veloce, risultati chiari

Feature	Min - Max
sepal length (cm)	4.3 - 7.9
sepal width (cm)	2.0 - 4.4
petal length (cm)	1.0 - 6.9
petal width (cm)	0.1 - 2.5

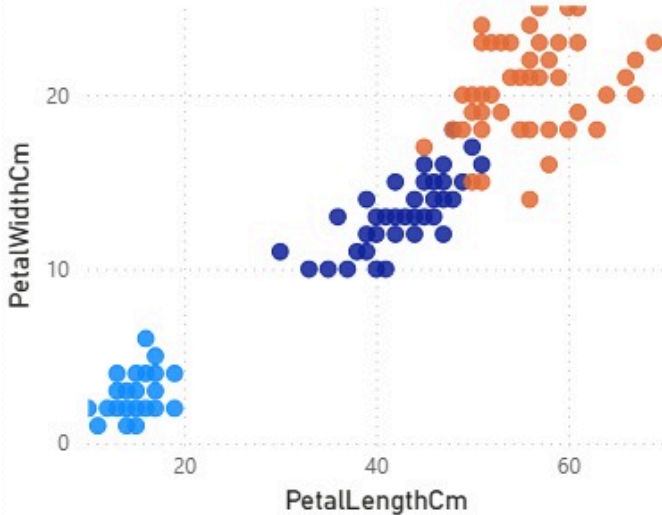
Dashboard Power BI

Analisi interattiva del Dataset Iris e risultati del modello

Dashboard

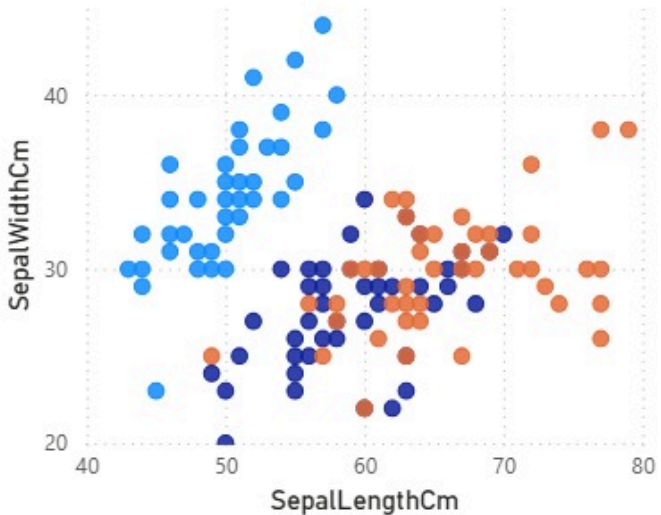
Species, PetalLengthCm e PetalWidthCm

Species ● Iris-setosa ● Iris-versicolor ● Iris-virginica



Species, SepalLengthCm e SepalWidthCm

Species ● Iris-setosa ● Iris-versicolor ● Iris-virginica



37,59

Media di PetalLengthCm

17,59

Deviazione standard di Pe...

11,99

Media di PetalWidthCm

7,61

Deviazione standard di P...

58,43

Media di SepalLengthCm

8,25

Deviazione standard di Se...

30,54

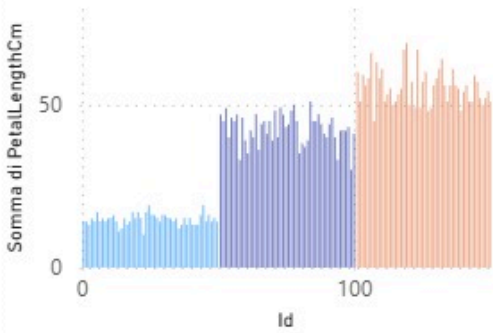
Media di SepalWidthCm

4,32

Deviazione standard di Se...

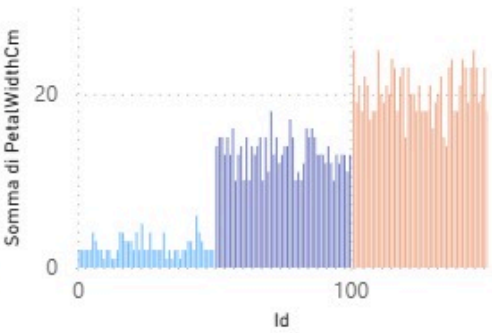
Somma di PetalLengthCm per Id e Species

Species ● Iris-setosa ● Iris-versic... ● Iris-virgin...



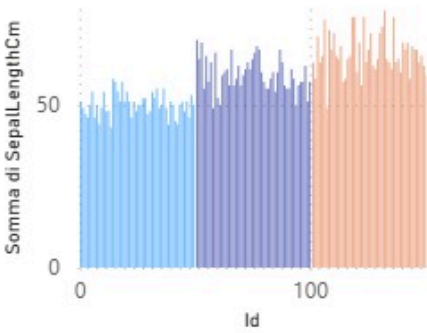
Somma di PetalWidthCm per Id e Species

Species ● Iris-setosa ● Iris-versic... ● Iris-virgin...



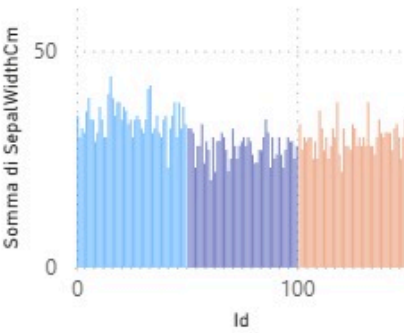
Somma di SepalLengthCm per Id e Species

Species ● Iris-setosa ● Iris-versicolor

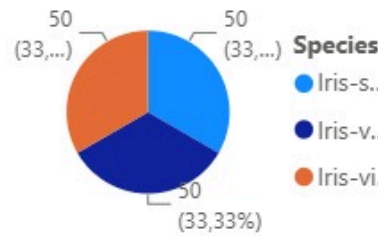


Somma di SepalWidthCm per Id e Species

Species ● Iris-setosa ● Iris-versico...



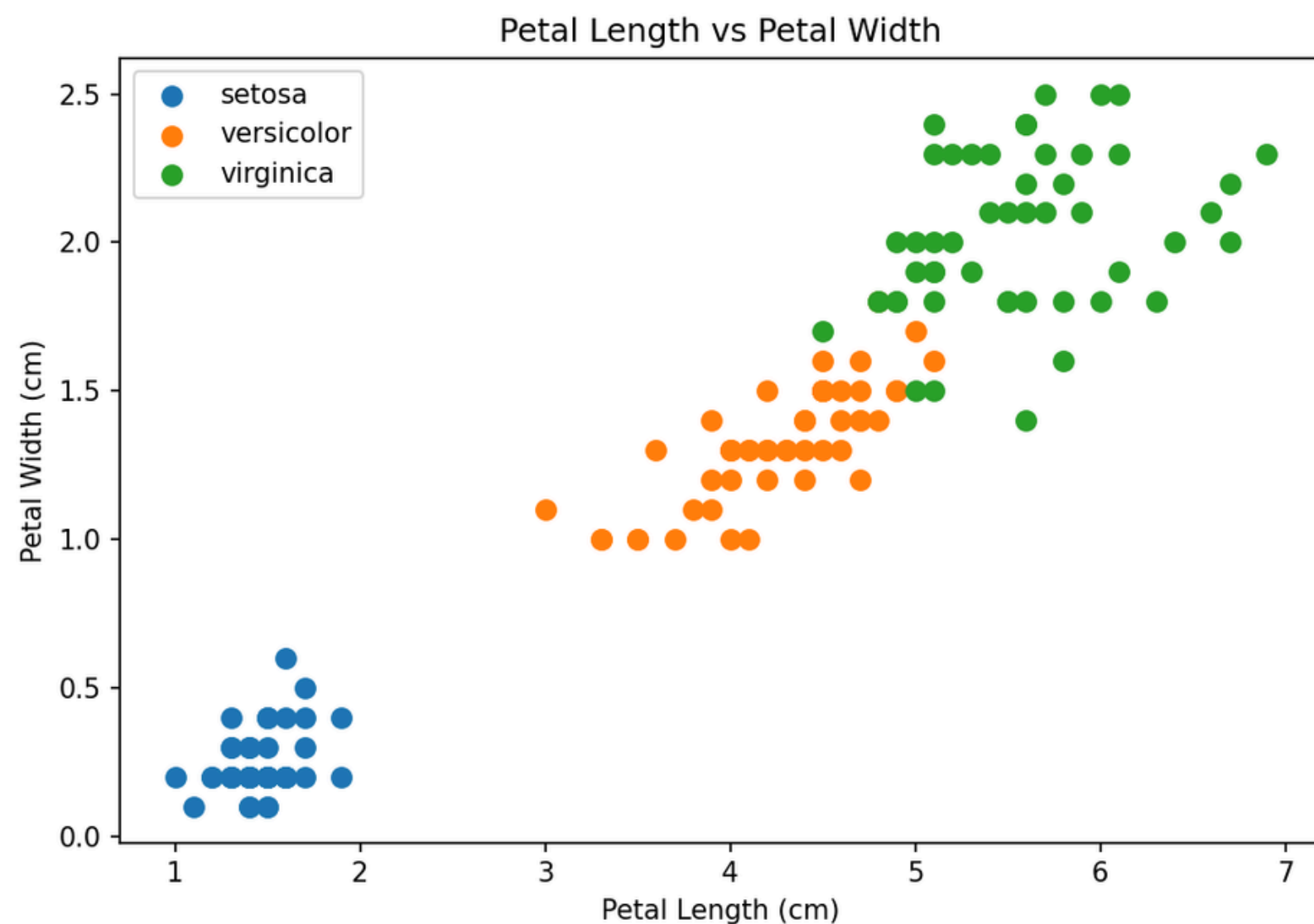
Conteggio di Id per Species



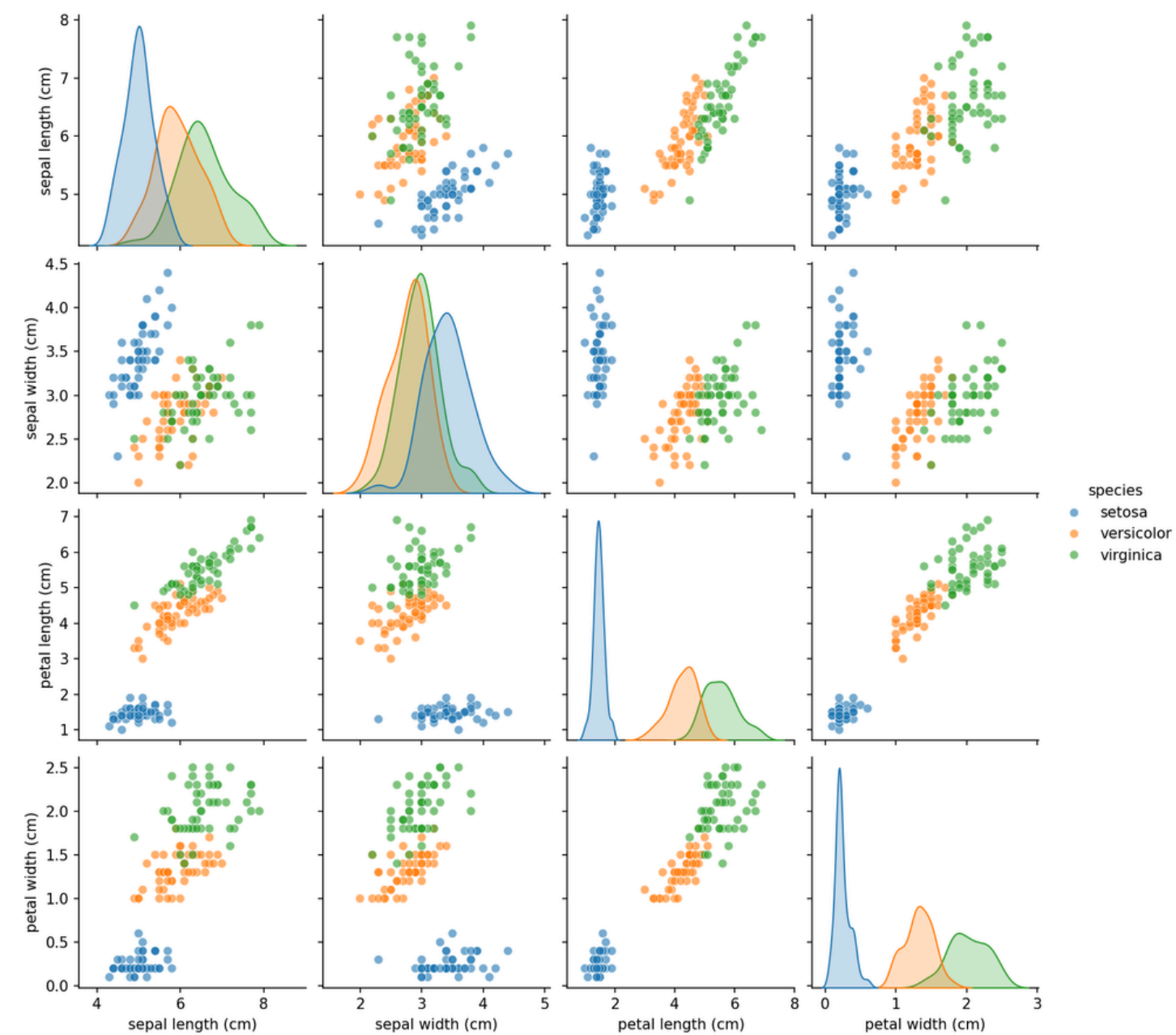
Abbiamo affiancato una dashboard Power BI alla presentazione per rendere l'analisi interattiva. A differenza delle slide statiche, permette di filtrare per specie, esplorare le distribuzioni in tempo reale e confrontare SGD vs Adam dinamicamente. Durante la sperimentazione ci ha permesso di confrontare configurazioni diverse senza rieseguire il notebook, e offre un'interfaccia accessibile a chiunque.

EDA — Scatter Plot & Pair Plot

Petal Length vs Width | Tutte le combinazioni di feature



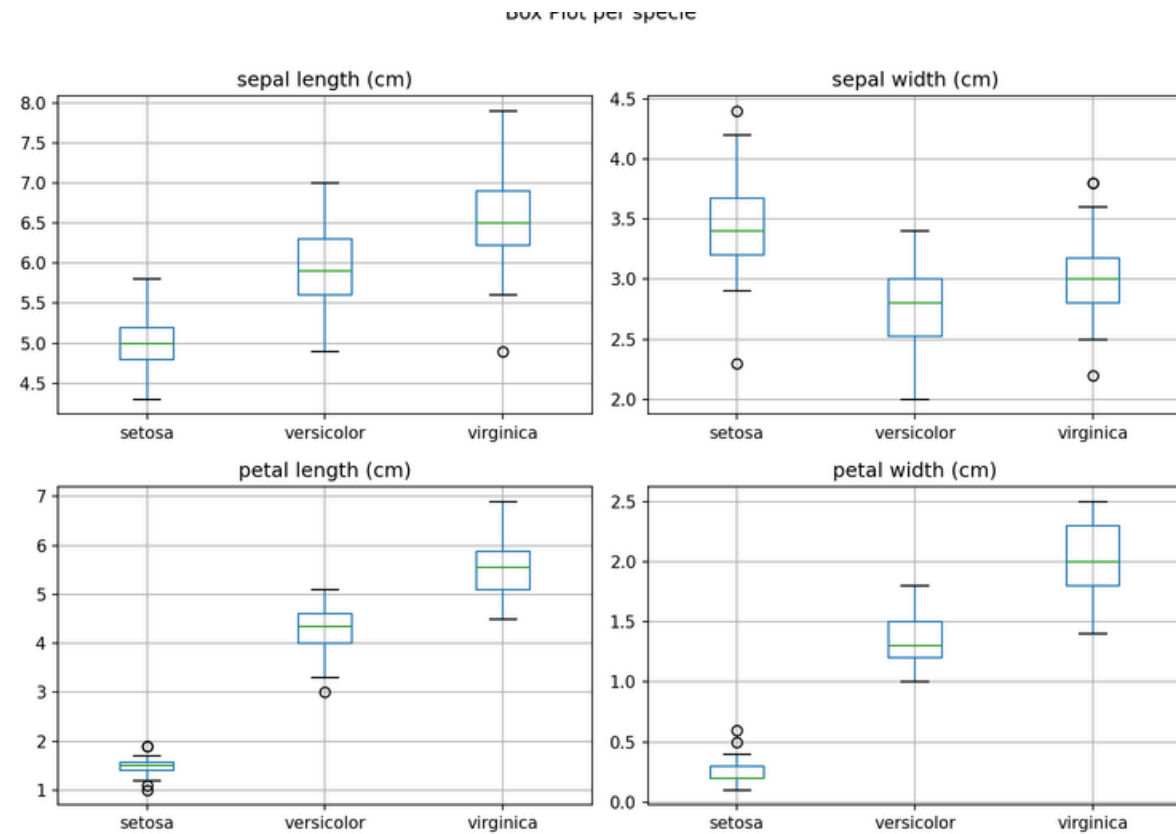
Setosa → cluster isolato in basso a sinistra
Versicolor e Virginica → quasi separabili con lieve sovrapposizione
Petal features separano molto meglio delle sepal features
Correlazione `petal_length / petal_width`: 0.96



Diagonale: KDE di ogni feature per specie
Fuori diagonale: scatter tra coppie di feature

Le righe/colonne di `petal_length` e `petal_width` mostrano la migliore separazione tra le 3 classi.

EDA — Box Plot Heatmap KDE



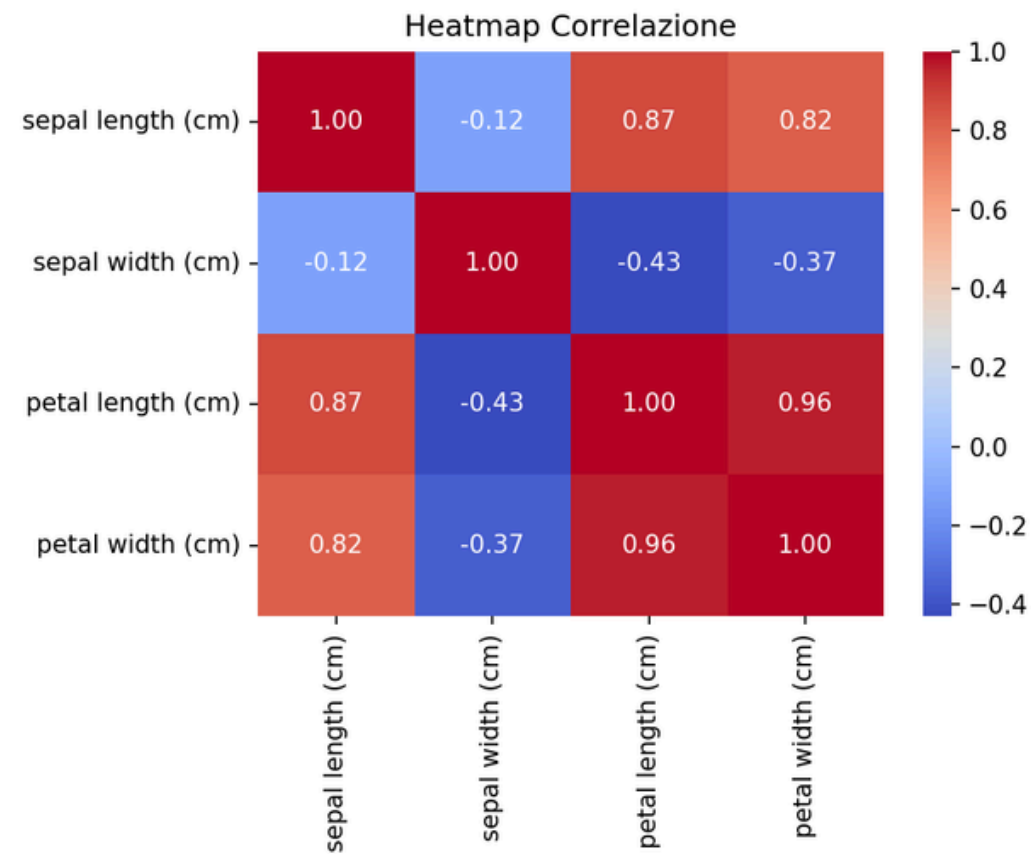
Linea centrale: mediana (50° percentile)

Box: 25° – 75° percentile (IQR)

Baffi: min/max escl. outlier

Cerchi: outlier

Setosa ha petal_length e petal_width nettamente inferiori alle altre due specie.

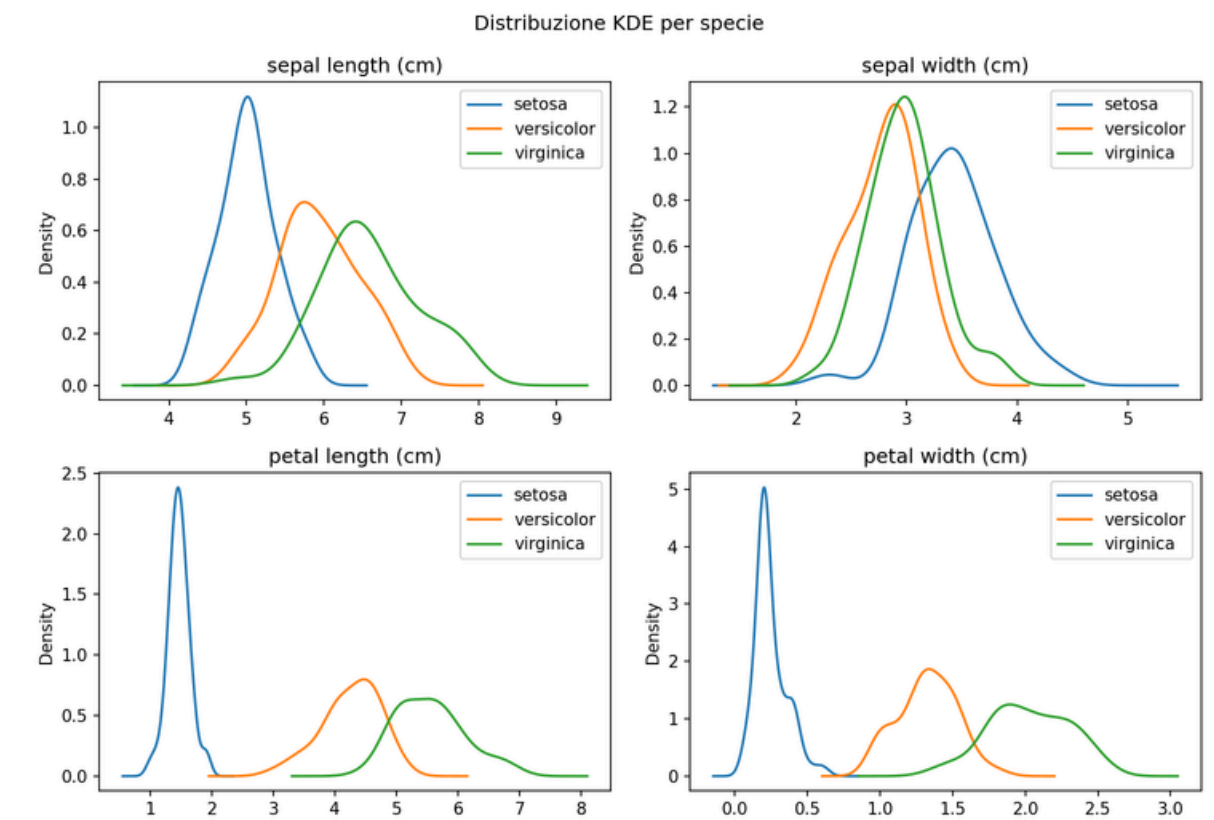


Heatmap: correlazione tra feature. Petal Length e Petal Width sono fortemente correlate (0.96).

Rosso (+1.0): alta correlazione positiva

Blu (-1.0): alta correlazione negativa

Bianco (0.0): nessuna correlazione



KDE — Kernel Density Estimate:

- Stima continua della distribuzione di probabilita

- Picchi separati: feature discriminativa

- Picchi sovrapposti: feature poco utile

- petal_length: picchi ben separati — ottima

- sepal_width: picchi sovrapposti — meno utile

Preprocessing dei Dati

1. StandardScaler

Normalizza: media=0, std=1

Perche: scale diverse confondono la rete

Fit SOLO su train
— no data leakage

2. OneHotEncoder

Converte classi in vettori binari

setosa [1,0,0]
versicolor [0,1,0]
virginica [0,0,1]

Evita ordinalita falsa ($0 < 1 < 2$)

Compatibile con Softmax
+ crossentropy

3. Train/Test Split 80/20

Train: 120 campioni
— aggiorna i pesi

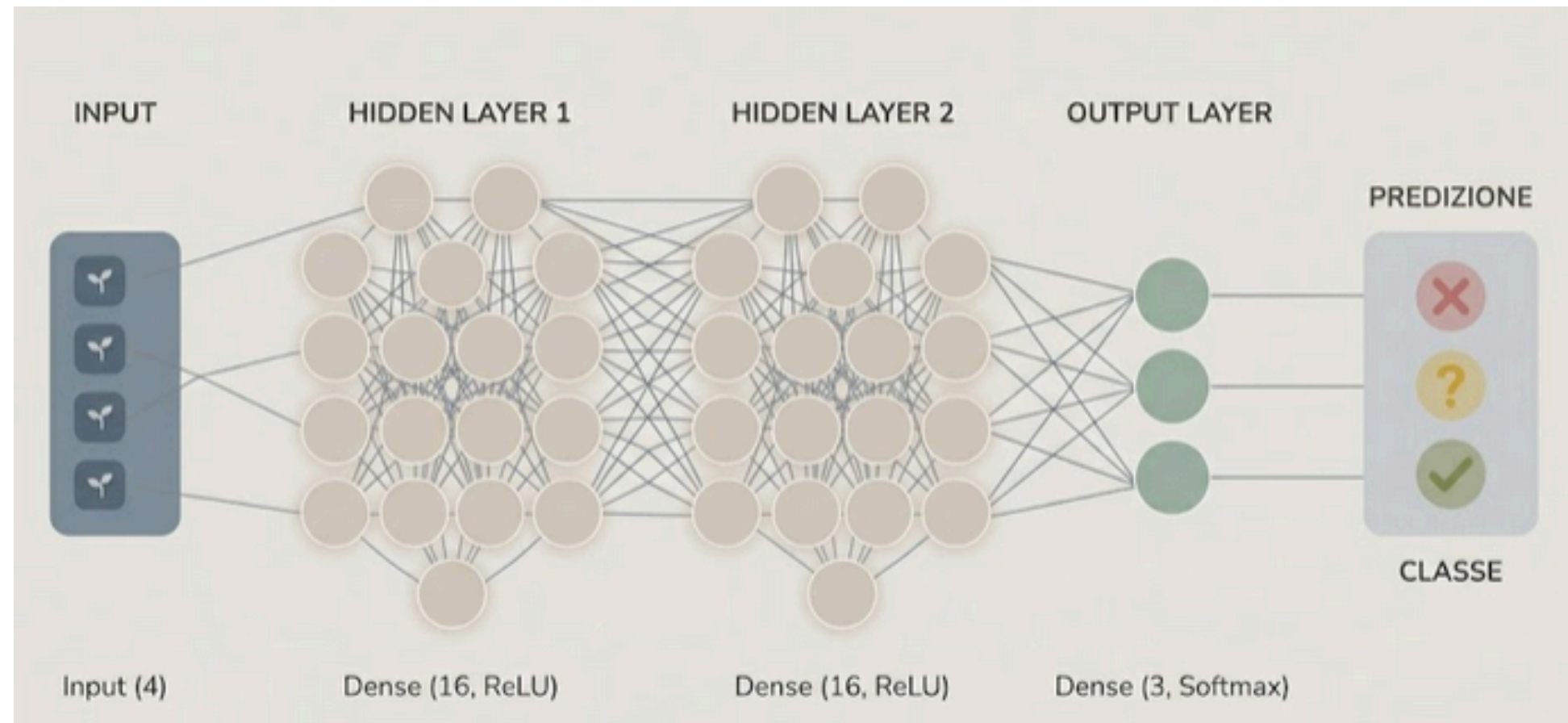
Test: 30 campioni
— solo valutazione finale

Non valutare mai su dati
di training

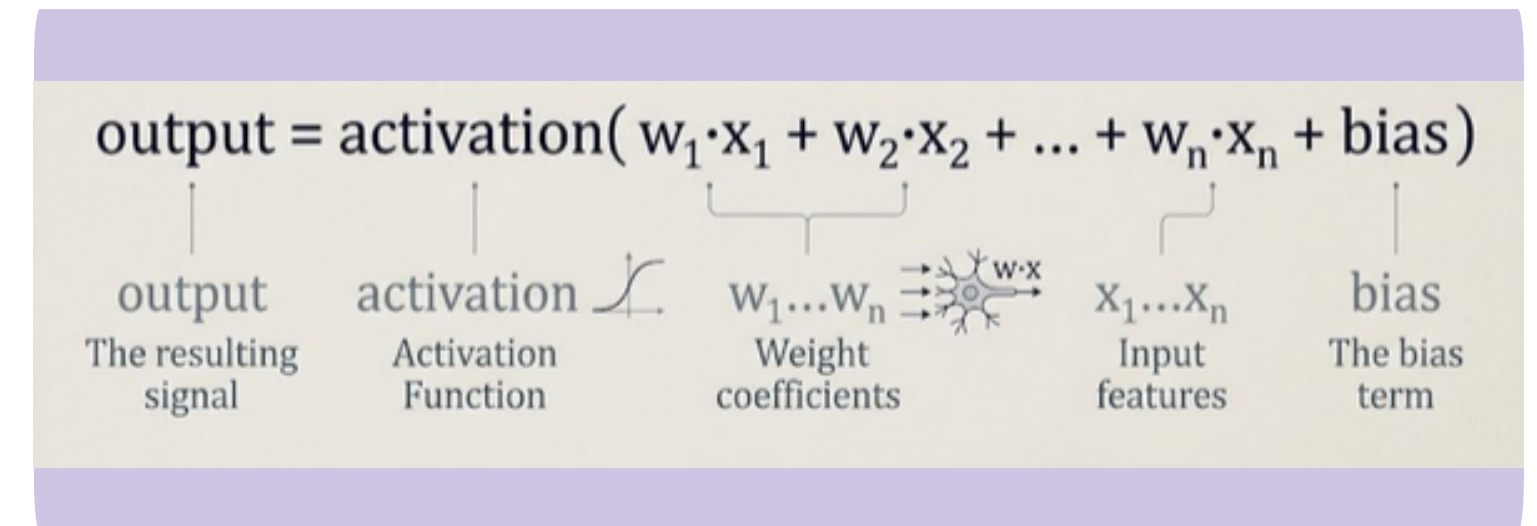
random_state=42
— riproducibile
stratify=y — classi bilanciate in ogni split

Teoria delle Reti Neurali

Architettura e funzionamento



Come funziona un neurone:



Come impara — 3 fasi per ogni batch:

1. Forward Pass: il dato attraversa tutti i layer e produce una predizione
2. Calcolo della Loss: si misura l'errore tra predizione e risposta corretta
3. Backpropagation: il gradiente indica come modificare ogni peso per ridurre l'errore

Scelta del numero di neuroni

Non esiste una formula — esistono regole pratiche

Situazione	Effetto sulla rete	Segnale nelle curve	Soluzione
Troppo pochi neuroni	Underfitting: non impara il pattern	train_acc bassa, val_acc bassa	Aumenta neuroni o aggiungi layer
Numero giusto	Generalizza correttamente	train_acc $\approx$ val_acc, entrambe alte	Mantieni questa configurazione
Troppi neuroni	Overfitting: memorizza i dati	train_acc alta, val_acc bassa	Riduci neuroni o aggiungi Dropout

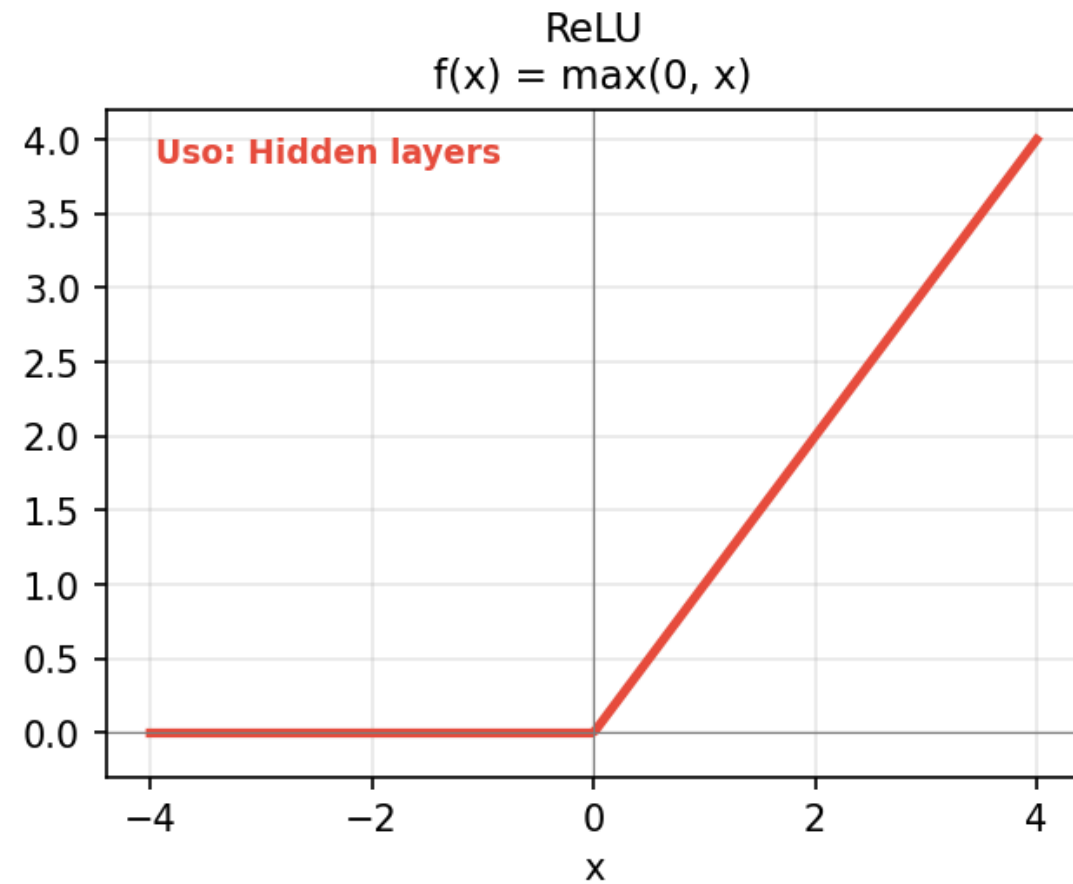
Regola empirica: inizia tra $n_{\text{feature}} \times 2$ e $n_{\text{feature}} \times 4$, poi aggiusta guardando le curve di training

Nel nostro caso — Dataset Iris:

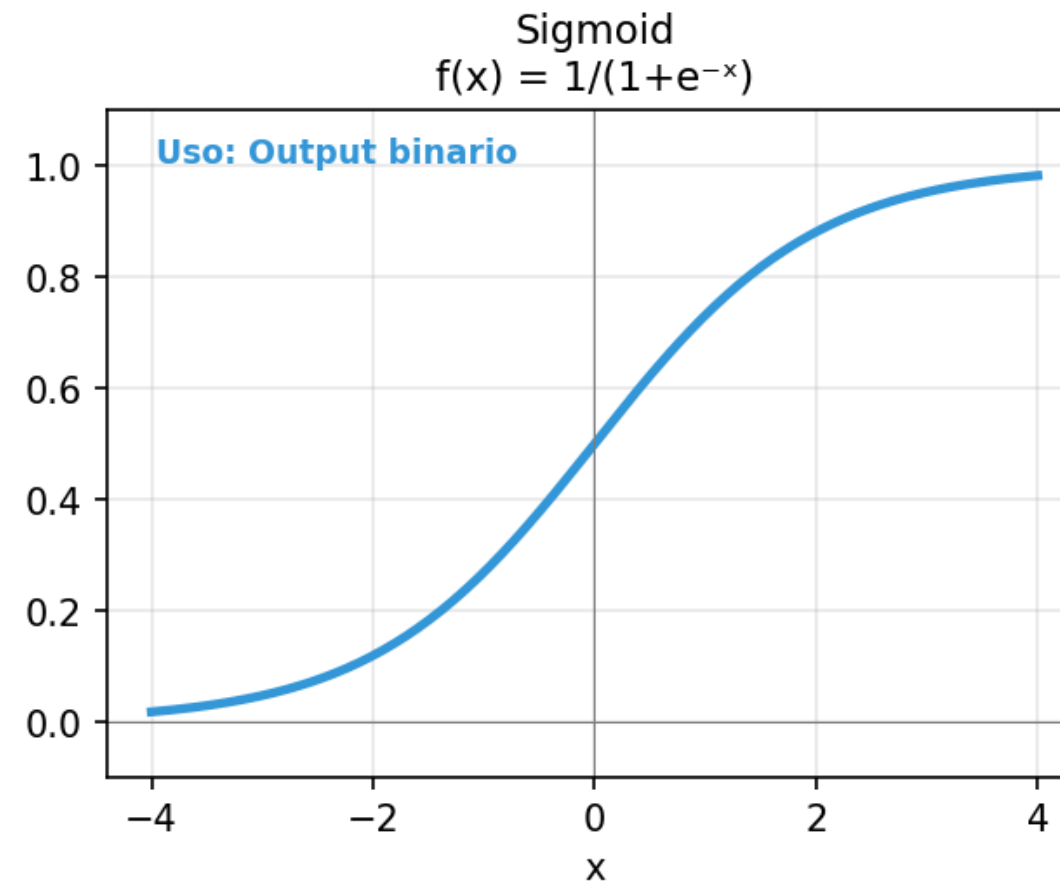
- 4 feature — range consigliato: 8-16 neuroni per layer
- Abbiamo scelto 16: abbastanza capiente per Iris, senza rischio overfitting su 150 campioni
- Con 8 neuroni funzionerebbe quasi uguale
- Con 128 rischieremmo di memorizzare il training set

Funzioni di Attivazione

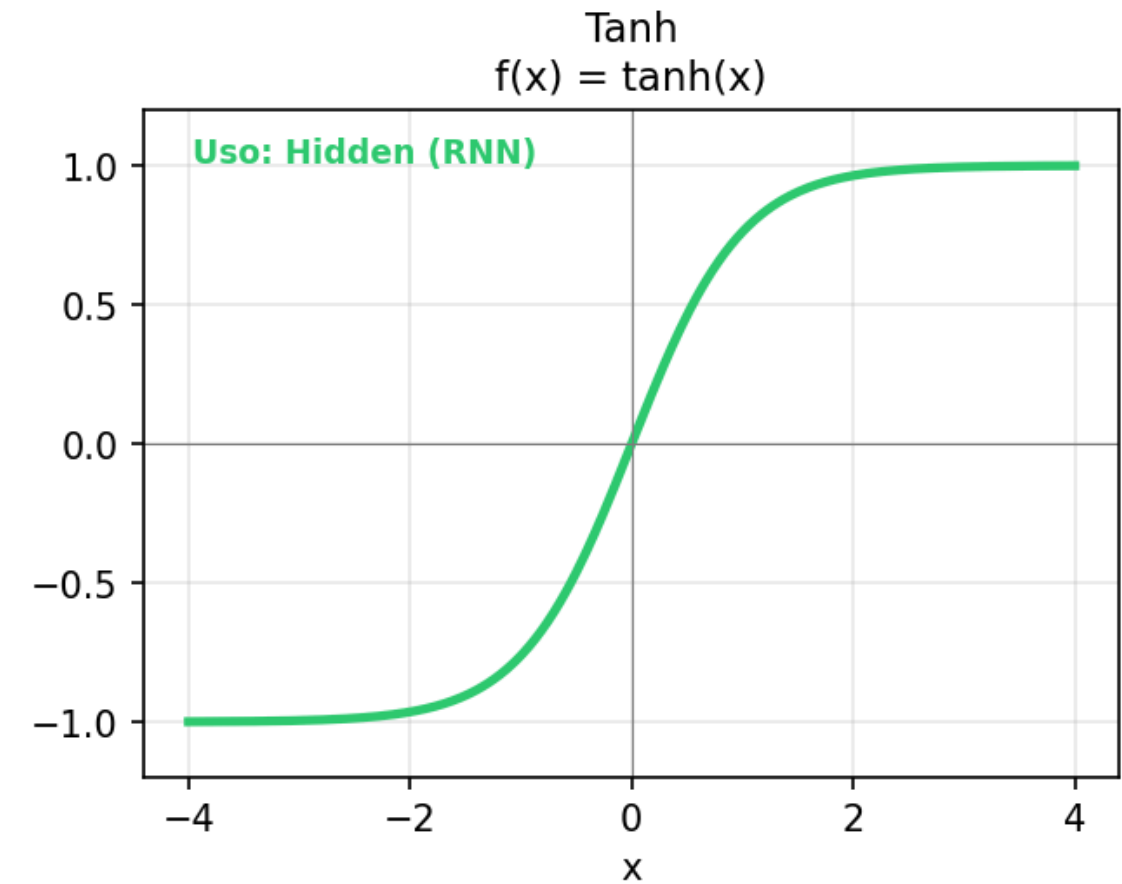
Introducono non-linearità — senza di esse la rete equivale a una regressione lineare



- Derivata = 1 per $x > 0$
- Nessun vanishing gradient
- Default per hidden layers
- Veloce da calcolare
- Problema: Dead ReLU (neuroni bloccati a 0)



- Output tra 0 e 1
- Derivata max = 0.25
- Vanishing gradient
- Usata per output binario
- Non per hidden layers nelle reti profonde

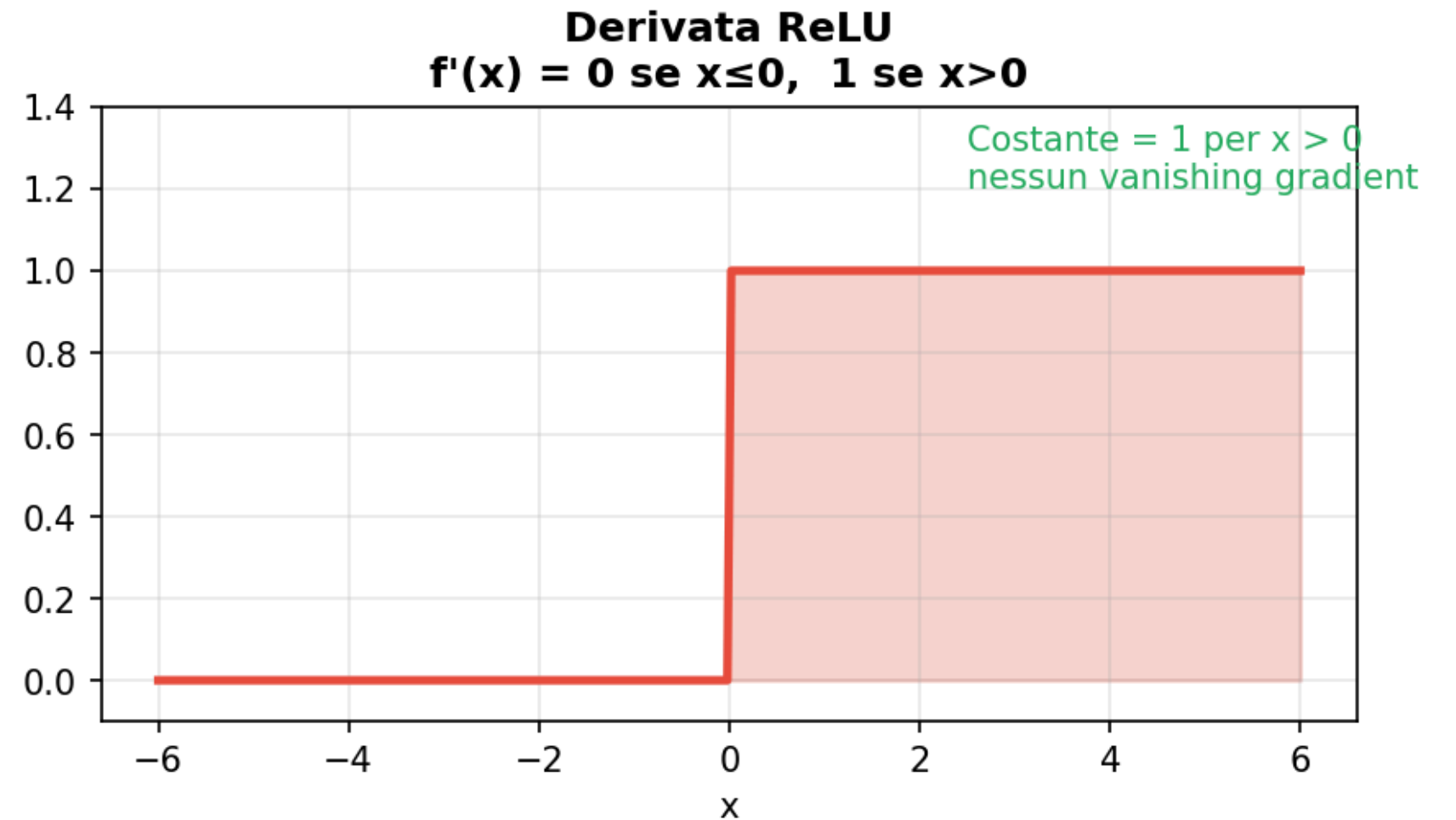
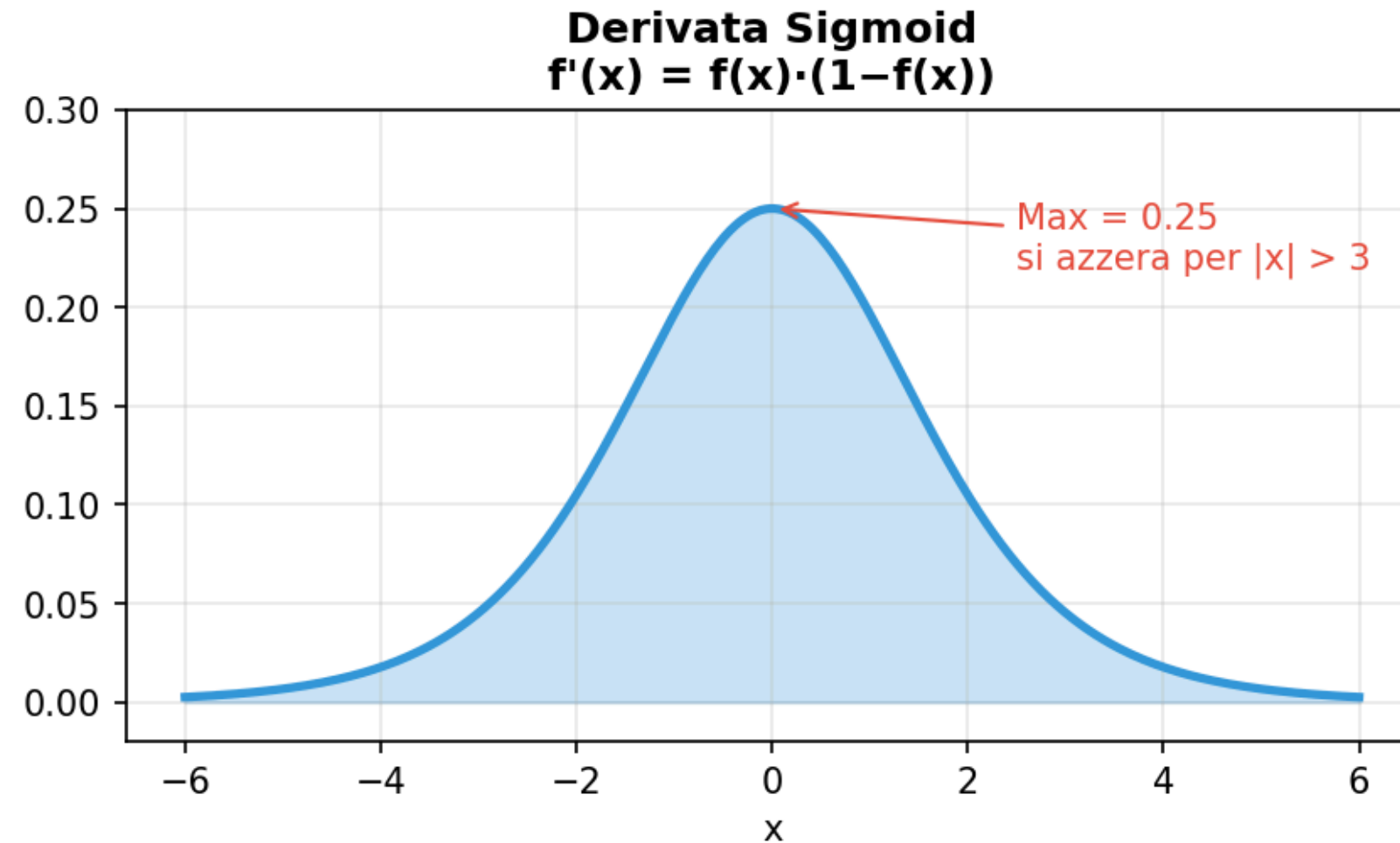


- Output tra -1 e 1
- Zero-centered
- Derivata max = 1.0
- Meglio di Sigmoid per hidden layers
- Ancora vanishing gradient

Vanishing Gradient

Perche ReLU e preferito negli hidden layer

Vanishing Gradient — perché ReLU batte Sigmoid negli hidden layer



Il problema:

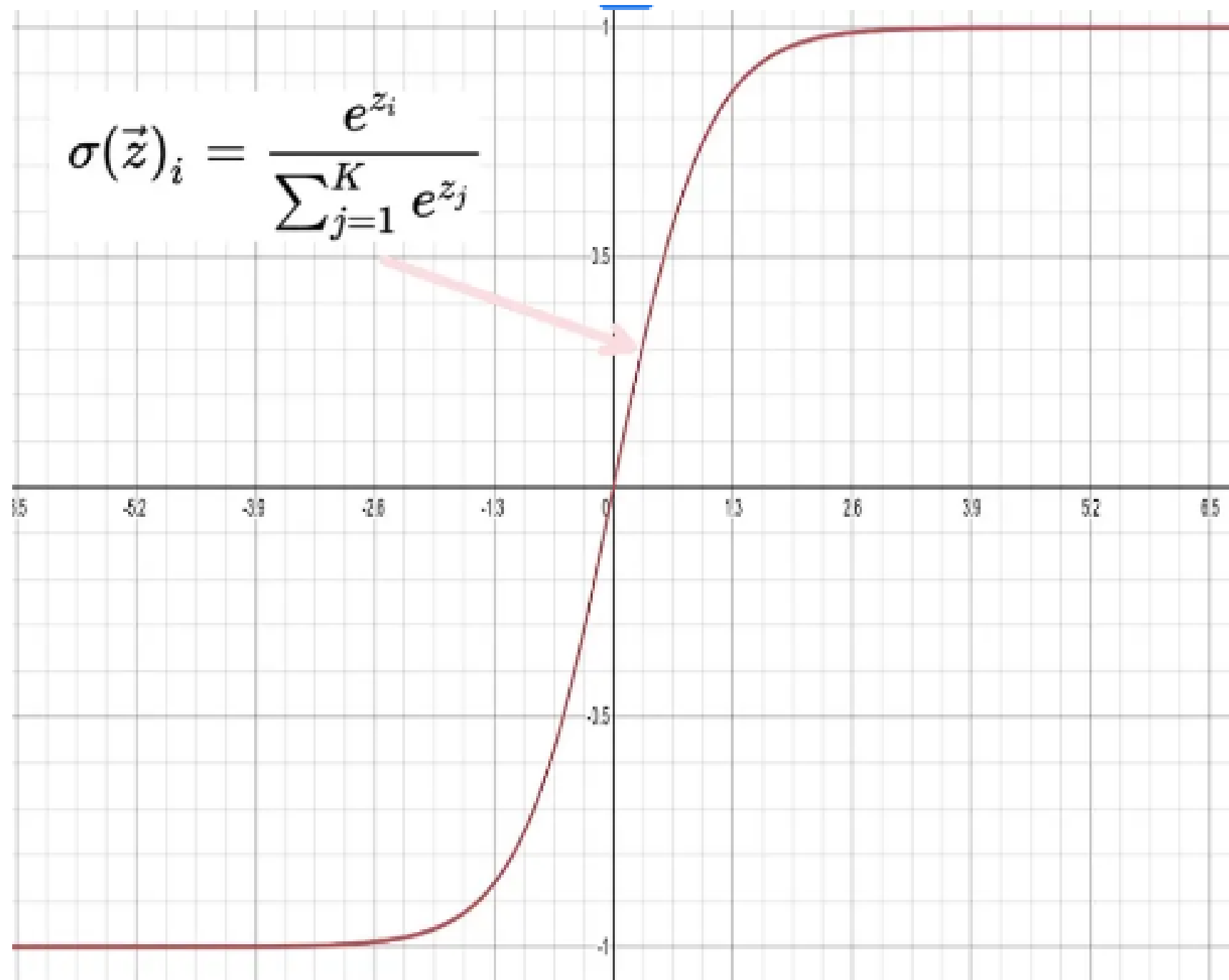
- Durante la backpropagation il gradiente si moltiplica per la derivata dell'activation ad ogni layer
- Sigmoid: derivata max = 0.25 — con 4 layer: $0.25^4 = 0.004$ — il gradiente si azzerà — i layer iniziali non imparano
- ReLU: derivata = 1 per $x > 0$ — il gradiente passa intatto — anche layer profondi imparano correttamente

Softmax — Attivazione dell'Output Layer

Standard per classificazione multi-classe

Perche Softmax?

- Produce una distribuzione di probabilita
— somma sempre = 1
- Amplifica le differenze: un valore leggermente piu alto diventa molto piu alto dopo Softmax — il modello impara a spingere il neurone corretto verso 1



SGD vs Adam — Formula in dettaglio

Stesso obiettivo, strategia di aggiornamento completamente diversa

SGD — Stochastic Gradient Descent

$$w = w - lr \times \text{gradiente}$$

- lr (learning rate): unico parametro controlla la dimensione del passo
- Uguale per TUTTI i pesi e per TUTTA la durata del training
- Problema: lr fisso non va bene per tutti i pesi
- lr troppo alto: oscilla, non converge
- lr troppo basso: converge lentissimo
- Trovare il lr giusto richiede trial & error

Adam — Adaptive Moment Estimation

$$\begin{aligned} m_t &= B1 \cdot m + (1-B1) \cdot \text{grad} \\ v_t &= B2 \cdot v + (1-B2) \cdot \text{grad}^2 \\ w &= w - lr \cdot m_{\text{hat}} / (\sqrt{v_{\text{hat}}} + \epsilon) \end{aligned}$$

- m = media mobile del gradiente (B1=0.9)
 - momentum
- v = media mobile del gradiente² (B2=0.999)
 - varianza
- Risultato: lr ADATTIVO per ogni peso
 - peso che cambia tanto: lr piccolo
 - peso che cambia poco: lr grande
- Non serve tuning del lr
 - i default funzionano quasi sempre
- Converte molto piu velocemente di SGD
- Usa piu memoria (salva m e v per ogni peso)

Nel nostro progetto: SGD acc=90.0% | Adam acc=96.67%— su Iris entrambi funzionano, su dataset grandi Adam e nettamente superiore

Costruzione del Modello con Keras

```
model = keras.Sequential([
    Input(shape=(4,)),
    Dense(16, activation='relu'),
    Dense(16, activation='relu'),
    Dense(3, activation='softmax'),
])

opt = SGD(learning_rate=0.01)
opt = Adam(learning_rate=0.001)

model.compile(
    optimizer=opt,
    loss='categorical_crossentropy',
    metrics=['accuracy'],
)
```

Scelte architetturali

2 hidden layer

Rappresentazioni più strutturate

16 neuroni / layer

Abbastanza per Iris senza overfitting

ReLU (hidden)

Veloce, no vanishing gradient, default sicuro

Softmax (output)

3 classi — distribuzione di probabilit 

categorical_crossentropy

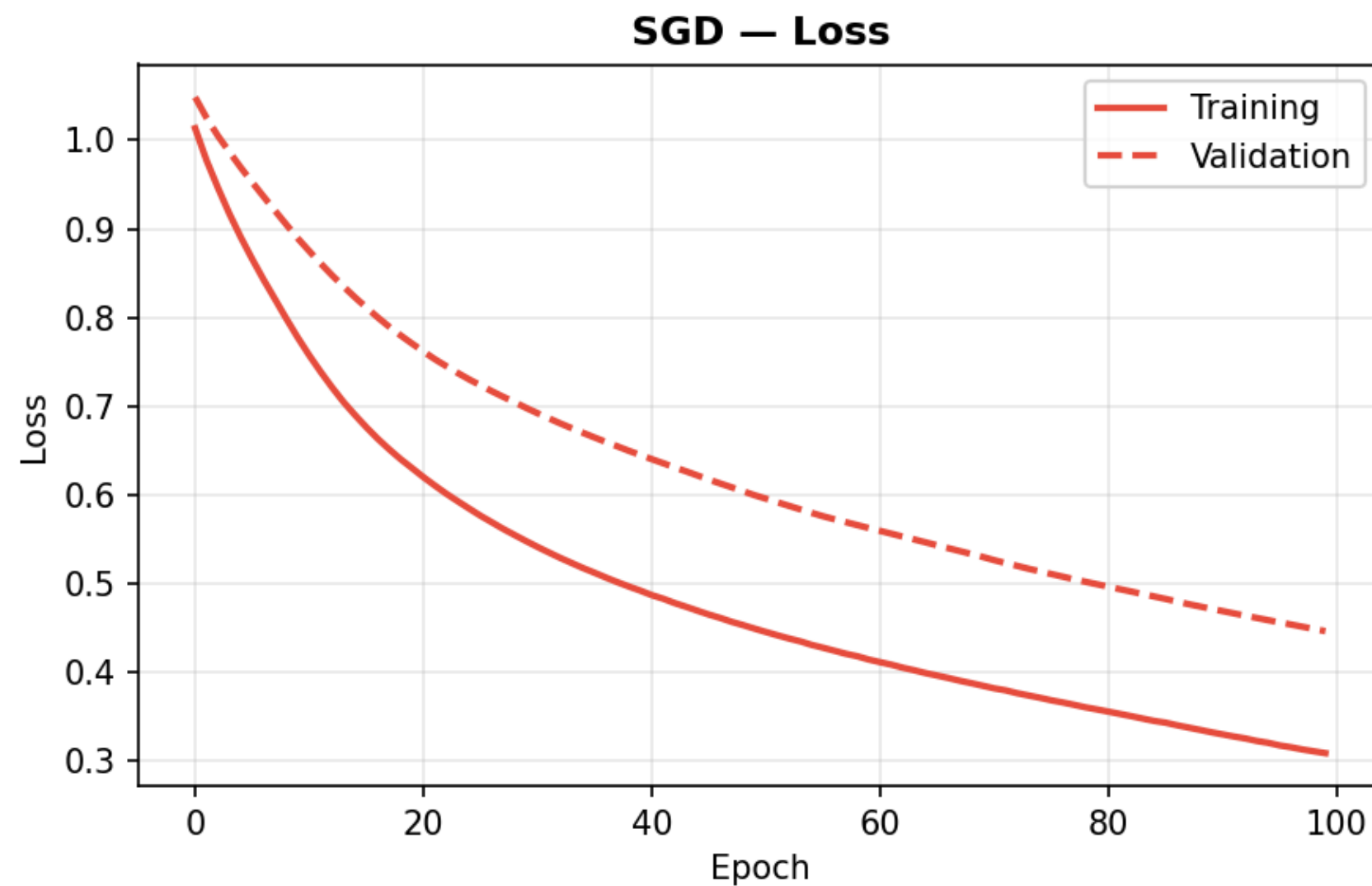
Standard multi-classe con one-hot encoding

403 parametri totali

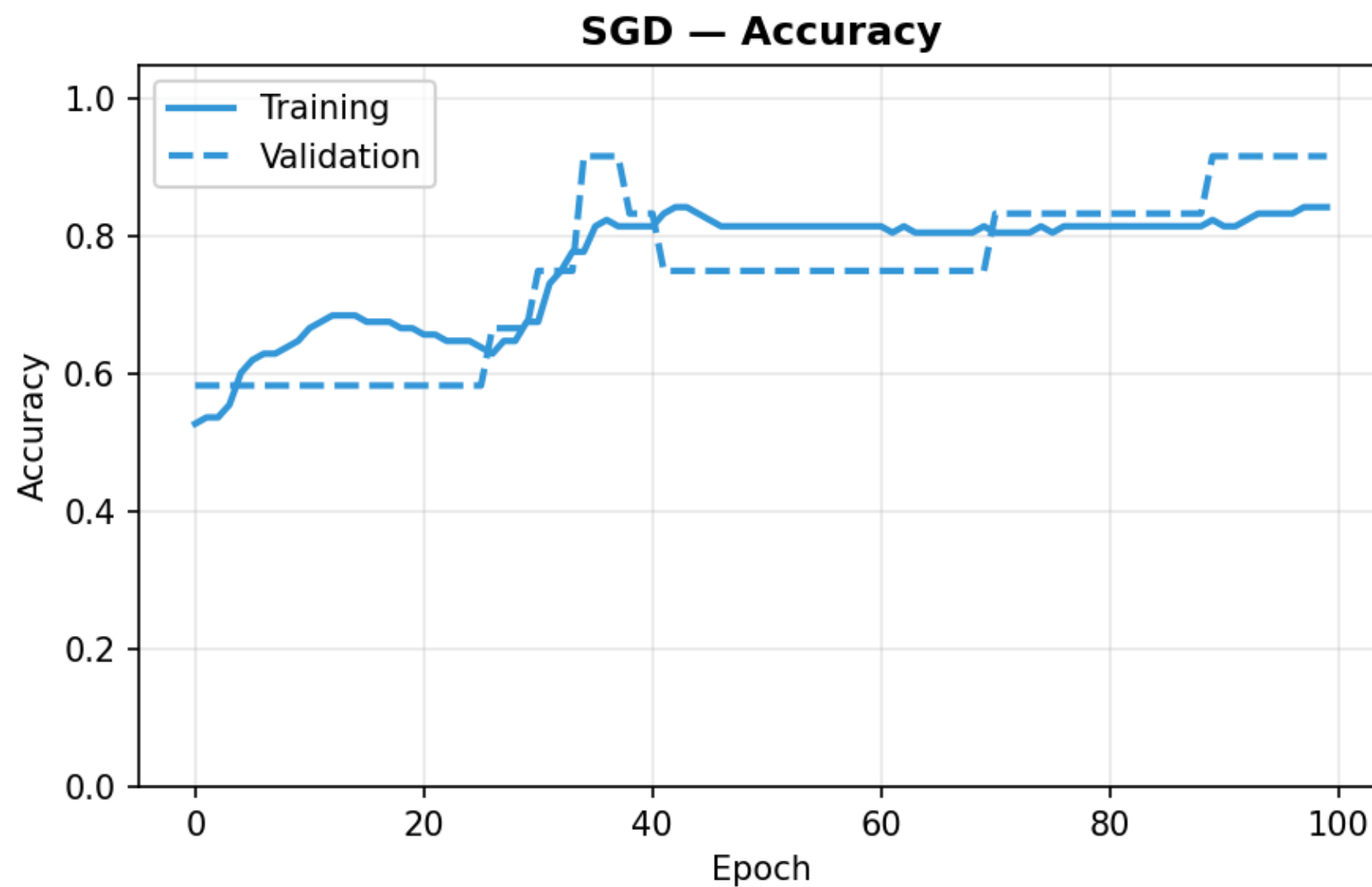
$4 \times 16 + 16 + 16 \times 16 + 16 + 16 \times 3 + 3 = 80 + 272 + 51$

Training e Risultati — SGD

Test Accuracy: 90.0% | Test Loss: 0.3045

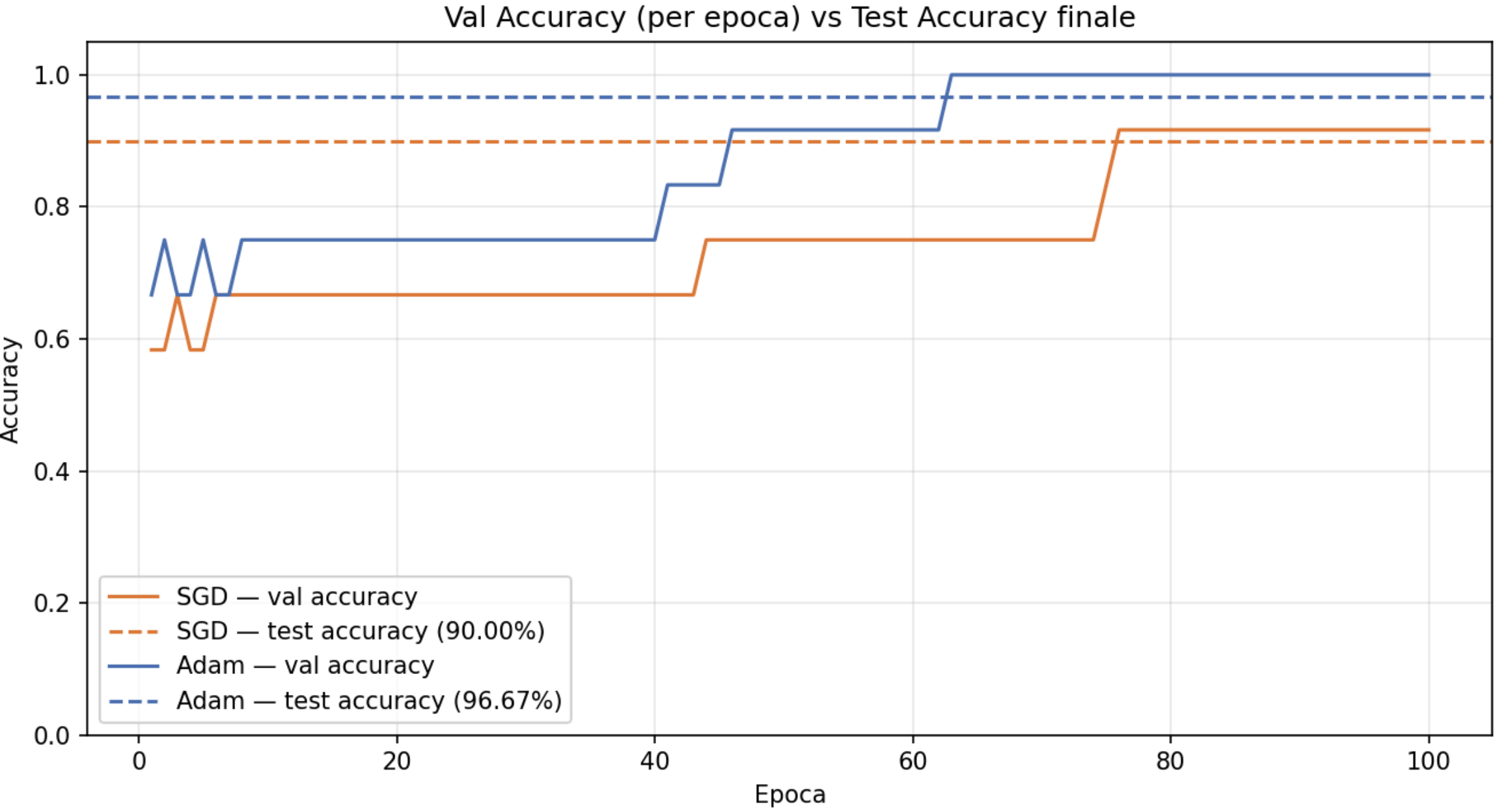


SGD — Loss (Training vs Validation)



SGD — Accuracy (Training vs Validation)

Confronto SGD vs Adam



Metrica	SGD	Adam
Test Accuracy	90.00%	96.67%
Test Loss	0.3045	0.1312

Grazie per l'attenzione

Andrea Morlotti · Gianluca Cerea

2025 - 2026